

passgen — Developer Reference

Single-file deterministic password generator (HMAC-SHA256)

Server: *app.py* (HTTP server, no dependencies)

Filena

Data Structures

CHK

Character set definitions. Array of [id, label, alphabet_string]. Used by renderChk(), gen().

```
const CHK = [  
  ['capitals', 'Zaglavnye', 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'],  
  ['lowercase', 'Strochnye', 'abcdefghijklmnopqrstuvwxyz'],  
  ['digits', 'Cifry', '0123456789'],  
  ['symbols', 'Specsimvoly', '!@#$%&*+=-.'],  
];
```

POPULAR

Predefined services with logo label and color. Used by renderSvcChips().

```
const POPULAR = [  
  {n:'google', l:'g', c:'#4285f4'},  
  {n:'github', l:'GH', c:'#333'},  
  {n:'vk', l:'V', c:'#4a76a8'},  
  {n:'yandex', l:'Ya', c:'#fc3f1d'},  
  {n:'mail', l:'@', c:'#005ff9'},  
  {n:'icloud', l:'i', c:'#369'},  
  {n:'dropbox', l:'D', c:'#0061ff'},  
  {n:'telegram', l:'T', c:'#0088cc'},  
];
```

Global State

Three mutable objects hold the application state:

```
let state = {capitals:1, lowercase:1, digits:0, symbols:1};  
let inv = {cap:0, dig:0};  
let svcs = [];
```

- state: toggle state for each character set (1=on, 0=off)
- inv: inversion options (cap=first uppercase, dig=last digit)
- svcs: user-added service names (sorted strings)

Functions — part 1

init()

Application entry point. Called at the end of the script.

```
function init(){
  renderChk();      // build checkbox grid
  renderInv();      // build inversion toggles
  loadState();      // restore from localStorage
  // Attach handlers:
  seed.addEventListener('input', () => { remove error class });
  seed.addEventListener('keydown', e => { if Enter -> gen() });
  service.addEventListener('keydown', e => { if Enter -> gen() });
}
```

To add a new input, attach its handler here in the same pattern.

renderChk()

Rebuilds the character-set checkbox panel from CHK array.

```
function renderChk(){
  const g=document.getElementById("chkGrid");
  g.innerHTML="";
  for(const[id,lbl]of CHK){
    const d=document.createElement("div");
    d.className="chk-item";
    d.onclick={()=>{ state[id]=+!state[id]; renderChk(); saveState(); }};
    // circle span + label text
    g.appendChild(d);
  }
}
```

Each click toggles state[id] between 0/1, re-renders, and saves.

renderInv()

Updates CSS classes on the two inversion toggle circles.

```
function renderInv(){
  invCap.className="chk-circle"+(inv.cap?" on":"");
  invDig.className="chk-circle"+(inv.dig?" on":"");
}
```

toggleInv(id)

Toggles an inversion option (cap or dig) and updates the UI.

```
function toggleInv(id){
  inv[id]=+!inv[id];
  document.getElementById('inv'+...).className = 'chk-circle' + (inv[id]?' on':'');
}
```

loadState() / saveState()

Persist/restore state to/from localStorage key "passgen_state".

```
function loadState(){
  const saved = localStorage.getItem('passgen_state');
  if(saved) {
    const d = JSON.parse(saved);
    state = d.state || state;
    inv  = d.inv  || inv;
    svcs = d.svcs || [];
    length.value = d.len || 10;
    renderChk(); renderInv(); renderSvcChips();
  }
}
```

```

    }
}

function saveState(){
  localStorage.setItem('passgen_state', JSON.stringify({
    state, svcs, len: +length.value
  }));
}

```

Seed is never saved. To persist a new setting, add it to both functions.

applyPreset()

Applies a preset configuration from the dropdown.

```

function applyPreset(){
  const v=preset.value;
  if(v==='custom')return;
  switch(v){
    case'min': state={...}; length.value=8; break;
    case'std': state={...}; length.value=10; break;
    case'max': state={...}; length.value=16; break;
  }
  renderChk();
}

```

adj(delta)

Adjusts the length input by delta, clamped to [1, 99].

```

function adj(d){
  const i=length;
  let v=+i.value+d; i.value = v<1 ? 1 : v>99 ? 99 : v;
}

```

Functions — part 2

exportData()

Downloads all settings (except seed) as a JSON file.

```
function exportData(){
  const data = { svcs, state, inv, len: +length.value };
  const blob = new Blob([JSON.stringify(data,null,2)], {type:'application/json'});
  const a = document.createElement('a');
  a.href = URL.createObjectURL(blob);
  a.download = 'passgen_export.json';
  a.click();
}
```

importData(e)

Reads a JSON file from the file input and restores state.

```
function importData(e){
  const file = e.target.files[0]; if(!file) return;
  const reader = new FileReader();
  reader.onload = () => {
    const d = JSON.parse(reader.result);
    if(d.svcs) { svcs = d.svcs; renderSvcChips(); }
    if(d.state) { state = d.state; renderChk(); }
    if(d.inv) { inv = d.inv; renderInv(); }
    if(d.len) { length.value = d.len; }
    saveState();
  };
  reader.readAsText(file);
}
```

toggleSvcPanel()

Toggles the service list panel (slide-down animation).

```
function toggleSvcPanel(){
  svcPanel.classList.toggle("open");
  svcToggle.classList.toggle("open");
}
```

CSS transition on max-height handles the animation.

renderSvcChips()

Renders popular and user-added service chips into the grid.

```
function renderSvcChips(){
  const g = document.getElementById('svcGrid'); g.innerHTML="";
  // Popular first (skip if already in svcs)
  for(const s of POPULAR)
    if(svcs.indexOf(s.n)===-1) g.appendChild(chip(s, true, false));
  // User-added after
  for(const s of svcs)
    g.appendChild(chip(s, false, true));
}
```

chip(item, hasLogo, deletable)

Creates a single service chip DOM element.

```
function chip(item, hasLogo, deletable){
  // item = string (user) or {n,l,c} (popular)
  const name = typeof item==="object" ? item.n : item;
  const d = document.createElement("div");
  d.className = "svc-chip";
```

```
// Logo dot or "?" placeholder
dot.style.cssText = '... background:' + (hasLogo ? item.c : 'var(--border)') + '...';
dot.textContent = hasLogo ? item.l : "?";
d.onclick = () => { service.value = name; close panel; };
if(deletable) {
  d.oncontextmenu = (e) => {
    e.preventDefault();
    svcs = svcs.filter(s => s !== name);
    renderSvcChips(); saveState();
  };
  d.title = 'PKM dlya udaleniya';
}
return d;
}
- Popular items: hasLogo=true, not deletable
- User-added items: hasLogo=false, deletable=true (right-click removes)
```

Functions — part 3

gen()

Main password generation function. Called by the "Create" button.

```
async function gen(){
  const seed = seedInput.value.trim();
  const svc = serviceInput.value.trim() || '(empty)';
  const len = +lengthInput.value || 10;
  // Validate seed
  if(!seed) { seedInput.classList.add("error"); seedErr.textContent="Vvedite sid"; return; }
  // Count enabled sets
  const sets = Object.values(state).filter(Boolean).length;
  if(!sets) { seedErr.textContent="Vyberite hotya by odin tip simvolov"; return; }
  // Build alphabet string from enabled CHK entries
  const alpha = CHK.filter(([id])=>state[id]).map(([_,l])=>l).join("");
  // Core HMAC generation
  const pwd = await hmacGen(seed, svc, len, alpha);
  // Apply inversion post-processing
  if(inv.cap && finalPwd.length>0)
    finalPwd = finalPwd[0].toUpperCase() + finalPwd.slice(1);
  if(inv.dig && finalPwd.length>0) {
    // Deterministic digit via HMAC(seed+"_lastdig")
    const digit = '0123456789'[dd[0] % 10];
    finalPwd = finalPwd.slice(0,-1) + digit;
  }
  // Display + copy + save service if new
  pwdField.value = finalPwd;
  updateStrength(len, sets);
  try { navigator.clipboard.writeText(finalPwd); } catch(e) {}
  if(!svcs.includes(svc) && svc!=='(empty)') {
    svcs.push(svc); svcs.sort(); renderSvcChips();
  }
  saveState();
}
```

hmacGen(seed, svc, len, alpha)

Core HMAC-SHA256 deterministic password generation. Uses rejection sampling for unbiased distribution.

```
async function hmacGen(seed, svc, len, alpha){
  const enc = new TextEncoder();
  const keyBytes = enc.encode(seed);
  const msgBase = enc.encode(svc + len);
  const al = alpha.length, max = 256 - (256 % al);
  let pwd = "", ct = 0;
  const k = await crypto.subtle.importKey(
    'raw', keyBytes, {name:'HMAC', hash:'SHA-256'}, false, ['sign']
  );
  while(pwd.length < len){
    const m = new Uint8Array(msgBase.length + 1);
    m.set(msgBase); m[msgBase.length] = ct++;
    const s = await crypto.subtle.sign('HMAC', k, m);
    for(const b of new Uint8Array(s))
      if(b < max) { pwd += alpha[b % al]; if(pwd.length === len) break; }
  }
  return pwd;
}
```

Key insight: $\text{max} = 256 - (256 \% \text{len}(\text{alpha}))$ ensures uniform distribution.

Counter increments each HMAC iteration to extend output.

Web Crypto API ensures the seed never leaves the browser sandbox.

genLogin()

Generates a short readable login from seed + service.

```
function genLogin(){
  const enc = new TextEncoder().encode(seed + 'login' + svc);
  crypto.subtle.digest('SHA-256', enc).then(buf => {
    const alpha = 'abcdefghijklmnopqrstuvwxyz23456789';
    const bytes = new Uint8Array(buf);
    let login = '';
    for(let i=0; i<6; i++)
      login += alpha[bytes[i] % alpha.length];
    loginField.value = login;
  });
}
```

Alphabet excludes ambiguous chars (l, o, 0, 1). Login is 6 chars for memorability.

Functions — part 4

updateStrength(len, sets)

Updates the strength indicator (3 bars + text) based on length and character set count.

```
function updateStrength(len, sets){
  let level, emoji, msg;
  if(len>=14 && sets>=4)
    { level='strong'; emoji='uwu'; msg='Otlichnyj parol' }
  else if(len>=10 && sets>=3)
    { level='medium'; emoji=':3'; msg='Horoshij, mozhno dlinnee' }
  else
    { level='weak'; emoji='>.<'; msg='Dobavte simvolov ili uvelichite dlinu' }
  // Render 3 bars; on bars get .on + level class for color
  // bar classes: .str-bar.on.weak(pink), .on.medium(crimson), .on.strong(lilac)
  info.className = "str-info " + level;
  info.textContent = emoji + " " + msg;
}
```

CSS colors: weak=var(--pink), medium=var(--crimson), strong=var(--lilac)

cpy()

Copies the generated password to the clipboard.

```
async function cpy(){
  const p = pwdField.value; if(!p) return;
  try { await navigator.clipboard.writeText(p); }
  catch { pwdField.select(); document.execCommand("copy"); }
  msg.textContent='Skopirovano!';
  setTimeout(() => msg.textContent="", 1500);
}
```

toggleAdvanced()

Expands/collapses the Advanced options panel (inversion toggles).

```
function toggleAdvanced(){
  const p = document.getElementById("advPanel");
  const a = document.getElementById("advArrow");
  if(p.style.maxHeight!='0px' && p.style.maxHeight!=""){
    p.style.maxHeight='0px'; p.style.opacity='0';
    a.style.transform='rotate(0deg)';
  }else{
    p.style.maxHeight='120px'; p.style.opacity='1';
    a.style.transform='rotate(90deg)';
  }
}
```

app.py — HTTP Server

Minimal Python HTTP server. No dependencies. Serves static files from the same directory.

```
#!/usr/bin/env python3
import http.server, socketserver, webbrowser, subprocess,
      shutil, threading, os, sys, signal
PORT = 8765
DIR = os.path.dirname(os.path.abspath(__file__))

class QuietHandler(http.server.SimpleHTTPRequestHandler):
    def log_message(self, fmt, *args): pass

def open_app_mode(url):
    # Tries Chromium/Brave/Vivaldi/Edge --app, falls back to Firefox,
```



```
# then webbrowser.open()
...

if __name__ == "__main__":
    httpd = TCPServer("", PORT), QuietHandler)
    # runs in background thread until Ctrl+C
```

To modify: change PORT, add endpoints, enable logs.

open_app_mode() uses --window-size=420,620 for the Chromium app window.

CSS Architecture

All colors in :root CSS variables:

```
:root {
  --bg: #f5f3ff;      --card: #fff;
  --fg: #1a1a2e;      --border: #e5e7eb;
  --muted: #999;      --shadow: rgba(90,20,140,.08);
  --pink: #f472b6;    --crimson: #e11d48;
  --lilac: #a855f7;    --lilac2: #7c3aed;
  --violet: #8b5cf6;  --error: #f43f5e;
  --hover: #ede9fe;   --chipbg: #ede9fe;
}
```

Animations:

- fadeUp: staggered card entrance
- glow: pulsing border on error
- slideDown/Up: service panel
- checkPop: toggle circle spring effect